Case Assignment #1

Sarthak Jain, MSIS 510 B

```
# Add imports
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf
```

```
# Load the datasets
df_train = pd.read_csv("train_data-1.csv")
df_test  = pd.read_csv("test_data-1.csv")
df_site_const = pd.read_csv("site_const_data-1-1.csv")
```

```
# Rename columns to replace '.' with '_' to use in smf methods
for df in (df_train, df_test, df_site_const):   # use your own dataframe names
    df.columns = df.columns.str.replace('.', '_', regex=False)
```

Q1

```
testing_size = df_test.shape[0]
training_size = df_train.shape[0]
print("Testing Set Size:", testing_size)
print("Training Set Size:", training_size)

total_sample_data = testing_size + training_size
print("Total Sample Size:", total_sample_data)

training_percentage = 100 * (training_size/total_sample_data)
testing_percentage =  100 * (testing_size/total_sample_data)

print("Training %:", training_percentage)
print("Testing %:", testing_percentage)
```

```
Testing Set Size: 85
Training Set Size: 374
Total Sample Size: 459
Training %: 81.48148148148148
Testing %: 18.51851851851852
```

Q2

```
# Linear Regression
linear_regression = smf.ols('annual_profit ~ agg_inc + sqft + col_grad + com60', data=df_train)
linear_regression_results = linear_regression.fit()

# Print model summary
print(linear_regression_results.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          annual_profit   R-squared:                       0.786
Model:                            OLS   Adj. R-squared:                  0.784
Method:                 Least Squares   F-statistic:                     339.1
Date:                Wed, 04 Nov 2026   Prob (F-statistic):          3.73e-122
Time:                        04:54:48   Log-Likelihood:                -5107.5
No. Observations:                 374   AIC:                         1.022e+04
Df Residuals:                     369   BIC:                         1.024e+04
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept    8.36e+04   4.11e+04      2.035      0.043    2810.065    1.64e+05
agg_inc        0.0028      0.000     20.288      0.000       0.003       0.003
sqft         383.3631     61.230      6.261      0.000     262.960     503.766
```

```
col_grad    3.468e+05   1.13e+05      3.069      0.002    1.25e+05    5.69e+05
com60       2.183e+05   9.82e+04      2.222      0.027    2.51e+04    4.11e+05
==============================================================================
Omnibus:                       56.244   Durbin-Watson:                   2.108
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               96.491
Skew:                           0.880   Prob(JB):                     1.11e-21
Kurtosis:                       4.759   Cond. No.                     1.82e+09
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.82e+09. This might indicate that there are
strong multicollinearity or other numerical problems.
```

```
print(linear_regression_results.params.apply(lambda x: f'{x:,.6f}'))
```

```
Intercept     83,597.092456
agg_inc           0.002807
sqft            383.363054
col_grad     346,821.138695
com60        218,265.764624
dtype: object
```

Q3

```
milagro_store_scenario = {
    'agg_inc': 100000000,
    'sqft': 800,
    'col_grad': 0.3,
    'com60': 0.1
}
annual_profitability = linear_regression_results.predict(milagro_store_scenario)
# print(annual_profitability)
print(f"Predicted profitability is: ${annual_profitability[0]:,.2f}")
```

```
Predicted profitability is: $796,894.99
```

Q4

```
# The R^2 for the training data. Need SSE and SST in order to calculate R^2
# a) R-squared value on training data
train_predictions = linear_regression_results.predict(df_train)
train_actual = df_train['annual_profit']

# Calculate SSE and SST for training data
train_mean = train_actual.mean()
train_sst = ((train_actual - train_mean) ** 2).sum()
train_sse = ((train_actual - train_predictions) ** 2).sum()

# Calculate R-squared for training data
train_r2 = 1 - (train_sse / train_sst)
print("Training data:")
print(f"R-squared: {train_r2:.4f}")


# The R^2 for the testing data. Need SSE and SST in order to calculate R^2
test_predictions = linear_regression_results.predict(df_test)
test_actual = df_test['annual_profit']

# Calculate SST and SSE for test data
test_sst = ((test_actual - train_mean) ** 2).sum()
test_sse = ((test_actual - test_predictions) ** 2).sum()

# Calculate R-squared for test data
test_r2 = 1 - (test_sse / test_sst)
print("Test data:")
print(f"R-squared: {test_r2:.4f}")
```

```
Training data:
R-squared: 0.7861
Test data:
R-squared: 0.7201
```

Q5

```
p_values = linear_regression_results.pvalues
print(p_values)

for feature, p_value in p_values.items():
    if p_value < 0.05:
        print(f"The feature '{feature}' is statistically significant (p-value: {p_value:.4f})")
    else:
        print(f"The feature '{feature}' is NOT statistically significant (p-value: {p_value:.4f})")

#Smallest p-value
smallest_p_value_feature = p_values.idxmin()
print(f"The feature with the smallest p-value is '{smallest_p_value_feature}' (p-value: {p_values[smallest_p_value_feature]:.4f
#Largest p-value
largest_p_value_feature = p_values.idxmax()
print(f"The feature with the largest p-value is '{largest_p_value_feature}' (p-value: {p_values[largest_p_value_feature]:.4f})"
```

```
Intercept    4.258413e-02
agg_inc      5.249611e-62
sqft         1.063204e-09
col_grad     2.309665e-03
com60        2.686647e-02
dtype: float64
The feature 'Intercept' is statistically significant (p-value: 0.0426)
The feature 'agg_inc' is statistically significant (p-value: 0.0000)
The feature 'sqft' is statistically significant (p-value: 0.0000)
The feature 'col_grad' is statistically significant (p-value: 0.0023)
The feature 'com60' is statistically significant (p-value: 0.0269)
The feature with the smallest p-value is 'agg_inc' (p-value: 0.0000)
The feature with the largest p-value is 'Intercept' (p-value: 0.0426)
```

Q6

```
exclude = ['state', 'store_number', 'annual_profit']
include = [col for col in df_train.columns if col not in exclude]

print("Features included in the model:")
for feature in include:
    print(f"  - {feature}")

print("Numerical predictor variables included in the model:")
for feature in include:
    if df_train[feature].dtype in ['int64', 'float64']:
        print(f"  - {feature}")
```

```
Features included in the model:
  - agg_inc
  - sqft
  - col_grad
  - com60
  - lci
  - nearcomp
  - nearmil
  - freestand
  - gini
  - housemed
Numerical predictor variables included in the model:
  - agg_inc
  - sqft
  - col_grad
  - com60
  - lci
  - nearcomp
  - nearmil
  - freestand
  - gini
  - housemed
```

```
correlation_matrix = df_train[include].corr()
print("Correlation matrix:")
print(correlation_matrix.round(5))
```

```
Correlation matrix:
           agg_inc     sqft  col_grad    com60      lci  nearcomp  nearmil  \
agg_inc    1.00000  0.51183   0.67019 -0.23883 -0.31483  -0.14758  0.16018
sqft       0.51183  1.00000   0.35303 -0.05535 -0.29966  -0.07415  0.12176
col_grad   0.67019  0.35303   1.00000 -0.22387 -0.31355  -0.18182  0.06533
com60     -0.23883 -0.05535  -0.22387  1.00000 -0.00408   0.02477  0.03771
lci       -0.31483 -0.29966  -0.31355 -0.00408  1.00000   0.07143 -0.11476
nearcomp  -0.14758 -0.07415  -0.18182  0.02477  0.07143   1.00000 -0.11045
nearmil    0.16018  0.12176   0.06533  0.03771 -0.11476  -0.11045  1.00000
freestand  0.17311  0.15047   0.16615 -0.03201 -0.21953   0.10422  0.21137
gini       0.06861  0.11095   0.00135 -0.03507 -0.10490   0.11227 -0.01217
housemed   0.48639  0.17686   0.54693 -0.10569 -0.15198  -0.17459  0.03186

           freestand     gini  housemed
agg_inc      0.17311  0.06861   0.48639
sqft         0.15047  0.11095   0.17686
col_grad     0.16615  0.00135   0.54693
com60       -0.03201 -0.03507  -0.10569
lci         -0.21953 -0.10490  -0.15198
nearcomp     0.10422  0.11227  -0.17459
nearmil      0.21137 -0.01217   0.03186
freestand    1.00000  0.02025  -0.01114
gini         0.02025  1.00000  -0.07971
housemed    -0.01114 -0.07971   1.00000
```

```
#The three pairs of variables with the strongest correlations (highest absolute values)
strongest_correlations = []

for i in range(len(correlation_matrix.columns)):
    for j in range(i+1, len(correlation_matrix.columns)):
        var1 = correlation_matrix.columns[i]
        var2 = correlation_matrix.columns[j]
        corr_value = correlation_matrix.iloc[i, j]
        strongest_correlations.append((var1, var2, abs(corr_value)))

# Sort by absolute correlation value in descending order and take the top 3
strongest_correlations.sort(key=lambda x: x[2], reverse=True)
top_3 = strongest_correlations[:3]

print("Three pairs of variables with the strongest correlations:")
for var1, var2, corr_value in top_3:
    print(f"  - {var1} and {var2}: {corr_value:.5f}")
```

```
Three pairs of variables with the strongest correlations:
  - agg_inc and col_grad: 0.67019
  - col_grad and housemed: 0.54693
  - agg_inc and sqft: 0.51183
```

Q7

```
complete_regression = smf.ols('annual_profit ~  agg_inc + col_grad + com60 + sqft + lci + nearcomp + nearmil + freestand + gini
print(complete_regression.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          annual_profit   R-squared:                       0.918
Model:                            OLS   Adj. R-squared:                  0.916
Method:                 Least Squares   F-statistic:                     406.1
Date:                Wed, 04 Nov 2026   Prob (F-statistic):          2.74e-190
Time:                        04:54:48   Log-Likelihood:                -4928.3
No. Observations:                 374   AIC:                             9879.
Df Residuals:                     363   BIC:                             9922.
Df Model:                          10
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept   1.279e+05    6.2e+04      2.062      0.040    5939.643     2.5e+05
agg_inc        0.0027   9.01e-05     29.854      0.000       0.003       0.003
col_grad    3.401e+05    7.7e+04      4.414      0.000    1.89e+05    4.92e+05
com60       1.825e+05   6.19e+04      2.951      0.003    6.09e+04    3.04e+05
sqft         294.6956     39.086      7.540      0.000     217.832     371.559
lci        -1.737e+04   4893.497     -3.549      0.000    -2.7e+04   -7742.203
nearcomp    3.131e+04   3360.382      9.317      0.000    2.47e+04    3.79e+05
```

```
nearmil     2642.5113    558.423      4.732      0.000    1544.361    3740.662
freestand   3.651e+05   2.07e+04     17.672      0.000    3.25e+05    4.06e+05
gini        1.819e+04   5.14e+04      0.354      0.724   -8.29e+04    1.19e+05
housemed     -15.0379     15.838     -0.949      0.343     -46.184      16.108
==============================================================================
Omnibus:                        7.389   Durbin-Watson:                   1.918
Prob(Omnibus):                  0.025   Jarque-Bera (JB):                9.371
Skew:                          -0.180   Prob(JB):                      0.00923
Kurtosis:                       3.687   Cond. No.                     2.07e+09
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.07e+09. This might indicate that there are
strong multicollinearity or other numerical problems.
```

```
complete_p_values = complete_regression.pvalues
print(complete_p_values)
for feature, p_value in complete_p_values.items():
    if p_value < 0.05:
        print(f"The feature '{feature}' is statistically significant (p-value: {p_value:.4f})")
    else:
        print(f"The feature '{feature}' is NOT statistically significant (p-value: {p_value:.4f})")
```

```
Intercept     3.989194e-02
agg_inc      9.111761e-100
col_grad      1.340048e-05
com60         3.372966e-03
sqft          3.808832e-13
lci           4.380442e-04
nearcomp      1.182668e-18
nearmil       3.188841e-06
freestand     7.175235e-51
gini          7.237427e-01
housemed      3.430107e-01
dtype: float64
The feature 'Intercept' is statistically significant (p-value: 0.0399)
The feature 'agg_inc' is statistically significant (p-value: 0.0000)
The feature 'col_grad' is statistically significant (p-value: 0.0000)
The feature 'com60' is statistically significant (p-value: 0.0034)
The feature 'sqft' is statistically significant (p-value: 0.0000)
The feature 'lci' is statistically significant (p-value: 0.0004)
The feature 'nearcomp' is statistically significant (p-value: 0.0000)
The feature 'nearmil' is statistically significant (p-value: 0.0000)
The feature 'freestand' is statistically significant (p-value: 0.0000)
The feature 'gini' is NOT statistically significant (p-value: 0.7237)
The feature 'housemed' is NOT statistically significant (p-value: 0.3430)
```

Q8

Model A: Kathleen's Original Model

```
model_a = smf.ols('annual_profit ~ agg_inc + sqft + col_grad + com60', data=df_train).fit()
print(model_a.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          annual_profit   R-squared:                       0.786
Model:                            OLS   Adj. R-squared:                  0.784
Method:                 Least Squares   F-statistic:                     339.1
Date:                Wed, 04 Nov 2026   Prob (F-statistic):          3.73e-122
Time:                        04:54:48   Log-Likelihood:                -5107.5
No. Observations:                 374   AIC:                         1.022e+04
Df Residuals:                     369   BIC:                         1.024e+04
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept    8.36e+04   4.11e+04      2.035      0.043    2810.065    1.64e+05
agg_inc        0.0028      0.000     20.288      0.000       0.003       0.003
sqft         383.3631     61.230      6.261      0.000     262.960     503.766
col_grad    3.468e+05   1.13e+05      3.069      0.002    1.25e+05    5.69e+05
com60       2.183e+05   9.82e+04      2.222      0.027    2.51e+04    4.11e+05
==============================================================================
Omnibus:                       56.244   Durbin-Watson:                   2.108
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               96.491
Skew:                           0.880   Prob(JB):                     1.11e-21
Kurtosis:                       4.759   Cond. No.                     1.82e+09
```

```
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.82e+09. This might indicate that there are
strong multicollinearity or other numerical problems.
```

```
model_a_train_r2 = model_a.rsquared

model_a_test_predictions = model_a.predict(df_test)
model_a_test_actual = df_test['annual_profit']
model_a_train_actual = df_train['annual_profit']
model_a_train_mean = model_a_train_actual.mean()

model_a_test_sst = ((model_a_test_actual - model_a_train_mean) ** 2).sum()
model_a_test_sse = ((model_a_test_actual - model_a_test_predictions) ** 2).sum()
model_a_test_r2 = 1 - (model_a_test_sse / model_a_test_sst)

    # Total predicted profitability for 48 construction sites
model_a_site_predictions = model_a.predict(df_site_const)
total_profit_millions_model_a = model_a_site_predictions.sum() / 1000000

print(f"Model A R-squared (training data): {model_a_train_r2:.4f}")
print(f"Model A R-squared (testing data): {model_a_test_r2:.4f}")
print(f"Total predicted profitability for 48 construction sites (Model A): ${total_profit_millions_model_a:,.2f} million")
```

```
Model A R-squared (training data): 0.7861
Model A R-squared (testing data): 0.7201
Total predicted profitability for 48 construction sites (Model A): $40.02 million
```

Model B: Full Model

```
model_b = smf.ols('annual_profit ~ agg_inc + col_grad + com60 + sqft + lci + nearcomp + nearmil + freestand + gini + housemed',
print(model_b.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          annual_profit   R-squared:                       0.918
Model:                            OLS   Adj. R-squared:                  0.916
Method:                 Least Squares   F-statistic:                     406.1
Date:                Wed, 04 Nov 2026   Prob (F-statistic):          2.74e-190
Time:                        04:54:48   Log-Likelihood:                -4928.3
No. Observations:                 374   AIC:                             9879.
Df Residuals:                     363   BIC:                             9922.
Df Model:                          10
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept   1.279e+05    6.2e+04      2.062      0.040    5939.643     2.5e+05
agg_inc        0.0027   9.01e-05     29.854      0.000       0.003       0.003
col_grad    3.401e+05    7.7e+04      4.414      0.000    1.89e+05    4.92e+05
com60       1.825e+05   6.19e+04      2.951      0.003    6.09e+04    3.04e+05
sqft         294.6956     39.086      7.540      0.000     217.832     371.559
lci        -1.737e+04   4893.497     -3.549      0.000    -2.7e+04   -7742.203
nearcomp    3.131e+04   3360.382      9.317      0.000    2.47e+04    3.79e+04
nearmil     2642.5113    558.423      4.732      0.000    1544.361    3740.662
freestand   3.651e+05   2.07e+04     17.672      0.000    3.25e+05    4.06e+05
gini        1.819e+04   5.14e+04      0.354      0.724   -8.29e+04    1.19e+05
housemed     -15.0379     15.838     -0.949      0.343     -46.184      16.108
==============================================================================
Omnibus:                        7.389   Durbin-Watson:                   1.918
Prob(Omnibus):                  0.025   Jarque-Bera (JB):                9.371
Skew:                          -0.180   Prob(JB):                      0.00923
Kurtosis:                       3.687   Cond. No.                     2.07e+09
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.07e+09. This might indicate that there are
strong multicollinearity or other numerical problems.
```

```
model_b_train_r2 = model_b.rsquared

model_b_test_predictions = model_b.predict(df_test)
model_b_test_actual = df_test['annual_profit']
model_b_train_actual = df_train['annual_profit']
model_b_train_mean = model_b_train_actual.mean()
```

```
model_b_test_sst = ((model_b_test_actual - model_b_train_mean) ** 2).sum()
model_b_test_sse = ((model_b_test_actual - model_b_test_predictions) ** 2).sum()
model_b_test_r2 = 1 - (model_b_test_sse / model_b_test_sst)

    # Total predicted profitability for 48 construction sites
model_b_site_predictions = model_b.predict(df_site_const)
total_profit_millions_model_b = model_b_site_predictions.sum() / 1000000

print(f"Model B R-squared (training data): {model_b_train_r2:.4f}")
print(f"Model B R-squared (testing data): {model_b_test_r2:.4f}")
print(f"Total predicted profitability for 48 construction sites (Model B): ${total_profit_millions_model_b:,.2f} million")
```

```
Model B R-squared (training data): 0.9179
Model B R-squared (testing data): 0.8272
Total predicted profitability for 48 construction sites (Model B): $36.06 million
```

Model C: Parsimonious Model

```
model_c = smf.ols('annual_profit ~ agg_inc + col_grad + com60 + sqft + lci + nearcomp + nearmil + freestand + gini + housemed',
print(model_c.summary())


# Non significant variables (p >= 0.05) from Model C
p_values_c = model_c.pvalues[1:]
non_significant = p_values_c[p_values_c >= 0.05].index.tolist()
print(f"\nNon-significant variables (p >= 0.05): {non_significant}")

# Identify highly correlated pairs (|correlation| > 0.70)
all_vars = ['agg_inc', 'sqft', 'col_grad', 'com60', 'lci', 'nearcomp', 'nearmil', 'freestand', 'gini', 'housemed']
corr_with_target = df_train[all_vars + ['annual_profit']].corr()['annual_profit'][all_vars]

print(f"\nCorrelations with annual_profit:")
for var in all_vars:
    print(f"  {var:12s}: {corr_with_target[var]:.4f}")

# Find pairs with |correlation| > 0.70
high_corr_pairs = []
for i, var1 in enumerate(all_vars):
    for var2 in all_vars[i+1:]:
        corr_val = df_train[[var1, var2]].corr().iloc[0, 1]
        if abs(corr_val) > 0.70:
            high_corr_pairs.append((var1, var2, corr_val))
            print(f"\nHigh correlation found: {var1} & {var2} = {corr_val:.4f}")
            print(f"  {var1} correlation with target: {corr_with_target[var1]:.4f}")
            print(f"  {var2} correlation with target: {corr_with_target[var2]:.4f}")

# Variables to remove
vars_to_remove = set(non_significant)

# For high correlation pairs, remove the one with weaker correlation to target
for var1, var2, _ in high_corr_pairs:
    if abs(corr_with_target[var1]) > abs(corr_with_target[var2]):
        vars_to_remove.add(var2)
        print(f"  → Removing {var2} (weaker correlation with target)")
    else:
        vars_to_remove.add(var1)
        print(f"  → Removing {var1} (weaker correlation with target)")

# Variables to keep
vars_to_keep = [v for v in all_vars if v not in vars_to_remove]
print(f"\nVariables removed: {sorted(vars_to_remove)}")
print(f"Variables kept in Model C: {vars_to_keep}")

# Build Model C
formula_c = 'annual_profit ~ ' + ' + '.join(vars_to_keep)
print(f"\nFormula: {formula_c}")

model_c = smf.ols(formula_c, data=df_train).fit()
print(model_c.summary())

# Evaluate Model C
model_c_train_r2 = model_c.rsquared

model_c_test_predictions = model_c.predict(df_test)
model_c_test_actual = df_test['annual_profit']
```

```
model_c_train_actual = df_train['annual_profit']
model_c_train_mean = model_c_train_actual.mean()

model_c_test_sst = ((model_c_test_actual - model_c_train_mean) ** 2).sum()
model_c_test_sse = ((model_c_test_actual - model_c_test_predictions) ** 2).sum()
model_c_test_r2 = 1 - (model_c_test_sse / model_c_test_sst)

    # Total predicted profitability for 48 construction sites
model_c_site_predictions = model_c.predict(df_site_const)
total_profit_millions_model_c = model_c_site_predictions.sum() / 1000000

print(f"Model C R-squared (training data): {model_c_train_r2:.4f}")
print(f"Model C R-squared (testing data): {model_c_test_r2:.4f}")
print(f"Total predicted profitability for 48 construction sites (Model C): ${total_profit_millions_model_c:,.2f} million")
```

```
[2] The condition number is large, 2.07e+09. This might indicate that there are
strong multicollinearity or other numerical problems.

Non-significant variables (p >= 0.05): ['gini', 'housemed']

Correlations with annual_profit:
  agg_inc     : 0.8683
  sqft        : 0.5791
  col_grad    : 0.6352
  com60       : -0.1496
  lci         : -0.4076
  nearcomp    : 0.0384
  nearmil     : 0.2606
  freestand   : 0.4850
  gini        : 0.0955
  housemed    : 0.3747

Variables removed: ['gini', 'housemed']
Variables kept in Model C: ['agg_inc', 'sqft', 'col_grad', 'com60', 'lci', 'nearcomp', 'nearmil', 'freestand']

Formula: annual_profit ~ agg_inc + sqft + col_grad + com60 + lci + nearcomp + nearmil + freestand
                            OLS Regression Results
==============================================================================
Dep. Variable:          annual_profit   R-squared:                       0.918
Model:                            OLS   Adj. R-squared:                  0.916
Method:                 Least Squares   F-statistic:                     508.7
Date:                Wed, 04 Nov 2026   Prob (F-statistic):          9.32e-193
Time:                        04:54:48   Log-Likelihood:                -4928.9
No. Observations:                 374   AIC:                             9876.
Df Residuals:                     365   BIC:                             9911.
Df Model:                           8
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept   1.256e+05   5.35e+04      2.348      0.019    2.04e+04    2.31e+05
agg_inc        0.0027   8.74e-05     30.581      0.000       0.003       0.003
sqft         299.4467     38.756      7.726      0.000     223.233     375.660
col_grad     3.13e+05   7.22e+04      4.337      0.000    1.71e+05    4.55e+05
com60       1.783e+05   6.16e+04      2.893      0.004    5.71e+04       3e+05
lci        -1.763e+04   4865.956     -3.624      0.000   -2.72e+04   -8063.427
nearcomp    3.167e+04   3327.214      9.519      0.000    2.51e+04    3.82e+04
nearmil     2647.7553    557.527      4.749      0.000    1551.387    3744.124
freestand   3.673e+05   2.05e+04     17.927      0.000    3.27e+05    4.08e+05
==============================================================================
Omnibus:                        7.427   Durbin-Watson:                   1.921
Prob(Omnibus):                  0.024   Jarque-Bera (JB):                9.432
Skew:                          -0.181   Prob(JB):                      0.00895
Kurtosis:                       3.689   Cond. No.                     1.96e+09
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.96e+09. This might indicate that there are
strong multicollinearity or other numerical problems.
Model C R-squared (training data): 0.9177
Model C R-squared (testing data): 0.8420
Total predicted profitability for 48 construction sites (Model C): $36.29 million
```

Model D: Alternative Model

```
model_d_best_option = ''
model_d_highest_profit = 0
model_d_highest_r2 = 0
```

Using lci

```
model_d_lci = smf.ols('annual_profit ~ agg_inc + col_grad + com60 + sqft + lci', data=df_train).fit()
print(model_d_lci.summary())

test_model_d_lci_pred = model_d_lci.predict(df_test)
test_model_d_lci_actual = df_test['annual_profit']
train_model_d_lci_mean = df_train['annual_profit'].mean()
test_model_d_lci_sst = ((test_model_d_lci_actual - train_model_d_lci_mean) ** 2).sum()
test_model_d_lci_sse = ((test_model_d_lci_actual - test_model_d_lci_pred) ** 2).sum()
test_model_d_lci_r2 = 1 - (test_model_d_lci_sse / test_model_d_lci_sst)

model_d_lci_site_pred = model_d_lci.predict(df_site_const)
model_d_lci_profit = model_d_lci_site_pred.sum() / 1000000

print(f"\nModel D LCI R-squared (testing data): {test_model_d_lci_r2:.4f}")
print(f"\nModel D LCI R-squared (training data): {model_d_lci.rsquared:.4f}")
print(f"Model D LCI Predicted Profitability: ${model_d_lci_profit:,.2f} million")

if test_model_d_lci_r2 > model_d_highest_r2:

    if model_d_lci_profit > model_d_highest_profit and model_d_lci_profit >= 40:
        model_d_best_option = 'lci'
        model_d_highest_r2 = test_model_d_lci_r2
        model_d_highest_profit = model_d_lci_profit
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          annual_profit   R-squared:                       0.797
Model:                            OLS   Adj. R-squared:                  0.794
Method:                 Least Squares   F-statistic:                     288.6
Date:                Wed, 04 Nov 2026   Prob (F-statistic):          6.06e-125
Time:                        04:54:48   Log-Likelihood:                -5097.9
No. Observations:                 374   AIC:                            1.021e+04
Df Residuals:                     368   BIC:                            1.023e+04
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept   3.905e+05   8.05e+04      4.851      0.000    2.32e+05    5.49e+05
agg_inc        0.0028      0.000     20.344      0.000       0.002       0.003
col_grad    2.719e+05   1.12e+05      2.436      0.015    5.24e+04    4.91e+05
com60        1.81e+05   9.62e+04      1.881      0.061   -8230.544     3.7e+05
sqft         340.0719     60.569      5.615      0.000     220.968     459.176
lci        -3.302e+04   7509.673     -4.397      0.000   -4.78e+04   -1.83e+04
==============================================================================
Omnibus:                       53.637   Durbin-Watson:                   2.091
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               91.347
Skew:                           0.847   Prob(JB):                     1.46e-20
Kurtosis:                       4.730   Cond. No.                     1.93e+09
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.93e+09. This might indicate that there are
strong multicollinearity or other numerical problems.

Model D LCI R-squared (testing data): 0.7670

Model D LCI R-squared (training data): 0.7968
Model D LCI Predicted Profitability: $40.10 million
```

Using nearcomp

```
model_d_nearcomp = smf.ols('annual_profit ~ agg_inc + col_grad + com60 + sqft + nearcomp', data=df_train).fit()
print(model_d_nearcomp.summary())

test_model_d_nearcomp_pred = model_d_nearcomp.predict(df_test)
test_model_d_nearcomp_actual = df_test['annual_profit']
train_model_d_nearcomp_mean = df_train['annual_profit'].mean()
test_model_d_nearcomp_sst = ((test_model_d_nearcomp_actual - train_model_d_nearcomp_mean) ** 2).sum()
test_model_d_nearcomp_sse = ((test_model_d_nearcomp_actual - test_model_d_nearcomp_pred) ** 2).sum()
test_model_d_nearcomp_r2 = 1 - (test_model_d_nearcomp_sse / test_model_d_nearcomp_sst)

model_d_nearcomp_site_pred = model_d_nearcomp.predict(df_site_const)
model_d_nearcomp_profit = model_d_nearcomp_site_pred.sum() / 1000000
```

```
print(f"\nModel D Nearcomp R-squared (testing data): {test_model_d_nearcomp_r2:.4f}")
print(f"Model D Nearcomp Predicted Profitability: ${model_d_nearcomp_profit:,.2f} million")

if test_model_d_nearcomp_r2 > model_d_highest_r2:

    if model_d_nearcomp_profit > model_d_highest_profit and model_d_nearcomp_profit >= 40:
        model_d_best_option = 'nearcomp'
        model_d_highest_r2 = test_model_d_nearcomp_r2
        model_d_highest_profit = model_d_nearcomp_profit
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          annual_profit   R-squared:                       0.818
Model:                            OLS   Adj. R-squared:                  0.815
Method:                 Least Squares   F-statistic:                     330.6
Date:                Wed, 04 Nov 2026   Prob (F-statistic):          1.09e-133
Time:                        04:54:49   Log-Likelihood:                -5077.4
No. Observations:                 374   AIC:                         1.017e+04
Df Residuals:                     368   BIC:                         1.019e+04
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept   -4.46e+04   4.12e+04     -1.083      0.280   -1.26e+05    3.64e+04
agg_inc        0.0028      0.000     22.235      0.000       0.003       0.003
col_grad    4.432e+05   1.05e+05      4.217      0.000    2.37e+05     6.5e+05
com60       2.339e+05   9.08e+04      2.577      0.010    5.54e+04    4.12e+05
sqft         381.0154     56.576      6.735      0.000     269.763     492.268
nearcomp    3.877e+04   4837.609      8.013      0.000    2.93e+04    4.83e+04
==============================================================================
Omnibus:                       82.792   Durbin-Watson:                   2.007
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              165.436
Skew:                           1.175   Prob(JB):                     1.19e-36
Kurtosis:                       5.257   Cond. No.                     1.84e+09
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.84e+09. This might indicate that there are
strong multicollinearity or other numerical problems.

Model D Nearcomp R-squared (testing data): 0.7703
Model D Nearcomp Predicted Profitability: $39.97 million
```

Using nearmil

```
model_d_nearmil = smf.ols('annual_profit ~ agg_inc + col_grad + com60 + sqft + nearmil', data=df_train).fit()
print(model_d_nearmil.summary())

test_model_d_nearmil_pred = model_d_nearmil.predict(df_test)
test_model_d_nearmil_actual = df_test['annual_profit']
train_model_d_nearmil_mean = df_train['annual_profit'].mean()
test_model_d_nearmil_sst = ((test_model_d_nearmil_actual - train_model_d_nearmil_mean) ** 2).sum()
test_model_d_nearmil_sse = ((test_model_d_nearmil_actual - test_model_d_nearmil_pred) ** 2).sum()
test_model_d_nearmil_r2 = 1 - (test_model_d_nearmil_sse / test_model_d_nearmil_sst)

model_d_nearmil_site_pred = model_d_nearmil.predict(df_site_const)
model_d_nearmil_profit = model_d_nearmil_site_pred.sum() / 1000000

print(f"\nModel D Nearmil R-squared (testing data): {test_model_d_nearmil_r2:.4f}")
print(f"Model D Nearmil Predicted Profitability: ${model_d_nearmil_profit:,.2f} million")

if test_model_d_nearmil_r2 > model_d_highest_r2:

    if model_d_nearmil_profit > model_d_highest_profit and model_d_nearmil_profit >= 40:
        model_d_best_option = 'nearmil'
        model_d_highest_r2 = test_model_d_nearmil_r2
        model_d_highest_profit = model_d_nearmil_profit
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          annual_profit   R-squared:                       0.800
Model:                            OLS   Adj. R-squared:                  0.797
Method:                 Least Squares   F-statistic:                     293.9
Date:                Wed, 04 Nov 2026   Prob (F-statistic):          4.22e-126
Time:                        04:54:49   Log-Likelihood:                -5095.2
No. Observations:                 374   AIC:                         1.020e+04
Df Residuals:                     368   BIC:                         1.023e+04
```

```
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept   3.773e+04   4.09e+04      0.924      0.356   -4.26e+04    1.18e+05
agg_inc        0.0027      0.000     20.068      0.000       0.002       0.003
col_grad    3.752e+05    1.1e+05      3.421      0.001     1.6e+05    5.91e+05
com60       1.844e+05   9.54e+04      1.933      0.054   -3211.098    3.72e+05
sqft         370.8839     59.384      6.246      0.000     254.110     487.658
nearmil     4214.5118    843.038      4.999      0.000    2556.735    5872.288
==============================================================================
Omnibus:                       20.166   Durbin-Watson:                   2.129
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               39.654
Skew:                           0.296   Prob(JB):                     2.45e-09
Kurtosis:                       4.481   Cond. No.                     1.82e+09
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.82e+09. This might indicate that there are
strong multicollinearity or other numerical problems.

Model D Nearmil R-squared (testing data): 0.7161
Model D Nearmil Predicted Profitability: $37.78 million
```

Using freestand

```
model_d_freestand = smf.ols('annual_profit ~ agg_inc + col_grad + com60 + sqft + freestand', data=df_train).fit()
print(model_d_freestand.summary())

test_model_d_freestand_pred = model_d_freestand.predict(df_test)
test_model_d_freestand_actual = df_test['annual_profit']
train_model_d_freestand_mean = df_train['annual_profit'].mean()
test_model_d_freestand_sst = ((test_model_d_freestand_actual - train_model_d_freestand_mean) ** 2).sum()
test_model_d_freestand_sse = ((test_model_d_freestand_actual - test_model_d_freestand_pred) ** 2).sum()
test_model_d_freestand_r2 = 1 - (test_model_d_freestand_sse / test_model_d_freestand_sst)

model_d_freestand_site_pred = model_d_freestand.predict(df_site_const)
model_d_freestand_profit = model_d_freestand_site_pred.sum() / 1000000

print(f"\nModel D Freestand R-squared (testing data): {test_model_d_freestand_r2:.4f}")
print(f"Model D Freestand Predicted Profitability: ${model_d_freestand_profit:,.2f} million")
if test_model_d_freestand_r2 > model_d_highest_r2:
    if model_d_freestand_profit > model_d_highest_profit and model_d_freestand_profit >= 40:
        model_d_best_option = 'freestand'
        model_d_highest_r2 = test_model_d_freestand_r2
        model_d_highest_profit = model_d_freestand_profit
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          annual_profit   R-squared:                       0.892
Model:                            OLS   Adj. R-squared:                  0.890
Method:                 Least Squares   F-statistic:                     605.5
Date:                Wed, 04 Nov 2026   Prob (F-statistic):          4.27e-175
Time:                        04:54:49   Log-Likelihood:                -4980.4
No. Observations:                 374   AIC:                             9973.
Df Residuals:                     368   BIC:                             9996.
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept   9.709e+04   2.93e+04      3.314      0.001    3.95e+04    1.55e+05
agg_inc        0.0027   9.88e-05     27.476      0.000       0.003       0.003
col_grad    2.425e+05   8.08e+04      3.003      0.003    8.37e+04    4.01e+05
com60       2.049e+05      7e+04      2.927      0.004    6.72e+04    3.43e+05
sqft         324.4291     43.758      7.414      0.000     238.383     410.476
freestand   4.241e+05   2.24e+04     18.926      0.000     3.8e+05    4.68e+05
==============================================================================
Omnibus:                        3.254   Durbin-Watson:                   1.985
Prob(Omnibus):                  0.196   Jarque-Bera (JB):                3.126
Skew:                           0.148   Prob(JB):                        0.210
Kurtosis:                       3.336   Cond. No.                     1.82e+09
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.82e+09. This might indicate that there are
strong multicollinearity or other numerical problems.
```

```
Model D Freestand R-squared (testing data): 0.8188
Model D Freestand Predicted Profitability: $37.33 million
```

```
print(f"\nModel D Best Option: {model_d_best_option}")
print(f"Highest R-squared: {model_d_highest_r2:.4f}")
print(f"Highest predicted profitability for 48 construction sites: ${model_d_highest_profit:,.2f} million")
```

```
Model D Best Option: lci
Highest R-squared: 0.7670
Highest predicted profitability for 48 construction sites: $40.10 million
```